

AI ASSISTANT CODING ASSIGNMENT-22

Name: A. Sumanth

Enrollment no: 2303A52191

Batch: 34

Semester: VI

Branch: CSE(AIML)

Lab 22 – Code Translation: Converting Between Programming Languages

Task 1 – Python to Java Conversion Task:

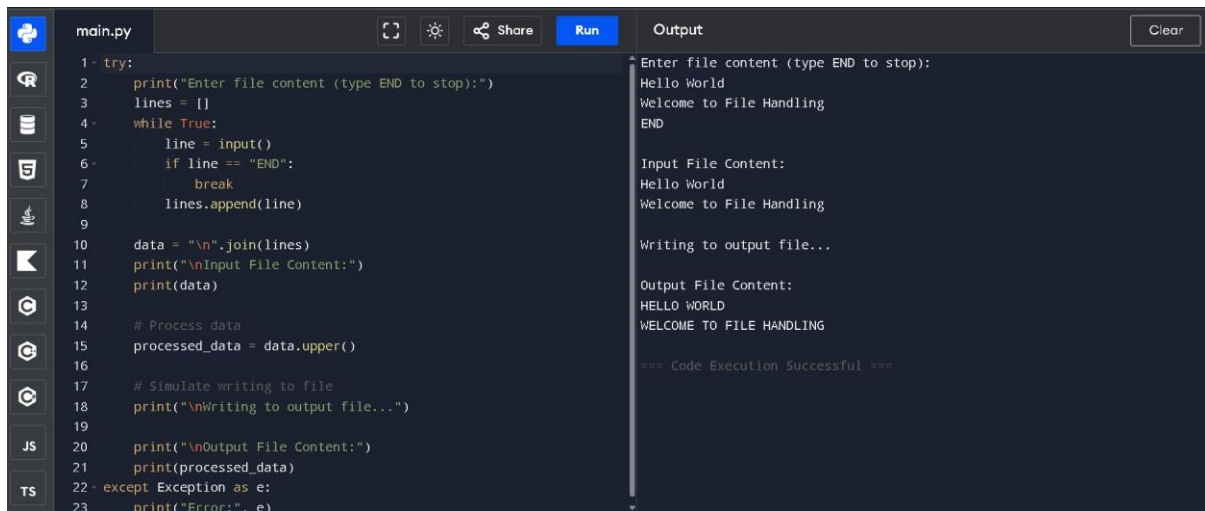
Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.



```
1- try:
2   print("Enter file content (type END to stop):")
3   lines = []
4   while True:
5       line = input()
6       if line == "END":
7           break
8       lines.append(line)
9
10  data = "\n".join(lines)
11  print("\nInput File Content:")
12  print(data)
13
14  # Process data
15  processed_data = data.upper()
16
17  # Simulate writing to file
18  print("\nWriting to output file...")
19
20  print("\nOutput File Content:")
21  print(processed_data)
22- except Exception as e:
23     print("Error:", e)
```

Output

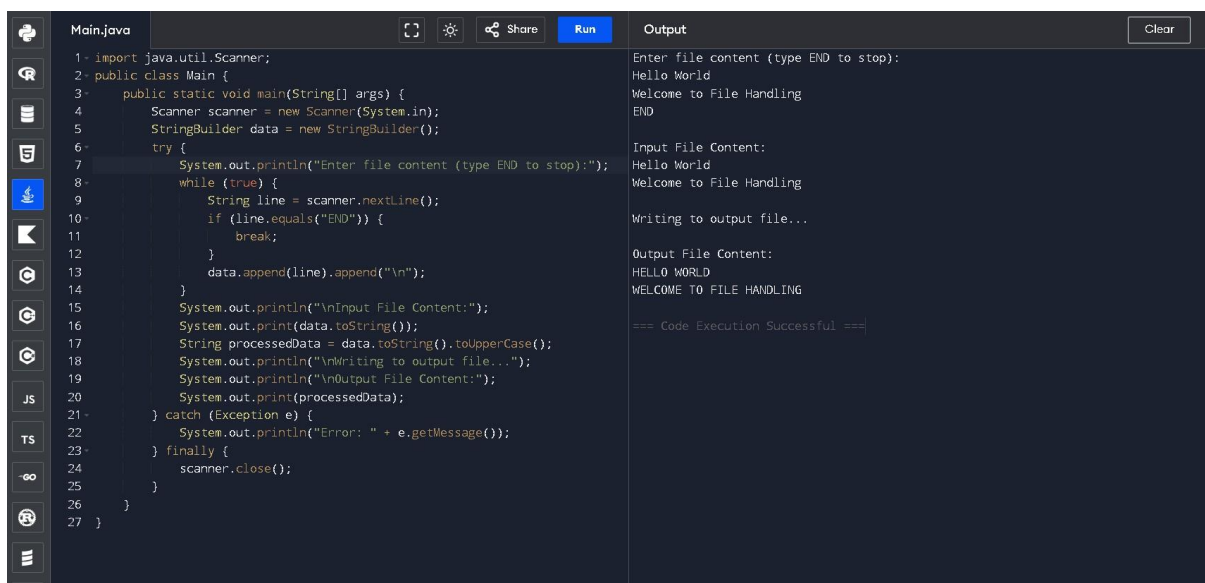
```
Enter file content (type END to stop):
Hello World
Welcome to File Handling
END

Input File Content:
Hello World
Welcome to File Handling

Writing to output file...

Output File Content:
HELLO WORLD
WELCOME TO FILE HANDLING

=== Code Execution Successful ===
```



```
1- import java.util.Scanner;
2- public class Main {
3-     public static void main(String[] args) {
4         Scanner scanner = new Scanner(System.in);
5         StringBuilder data = new StringBuilder();
6         try {
7             System.out.println("Enter file content (type END to stop):");
8             while (true) {
9                 String line = scanner.nextLine();
10                if (line.equals("END")) {
11                    break;
12                }
13                data.append(line).append("\n");
14            }
15            System.out.println("\nInput File Content:");
16            System.out.print(data.toString());
17            String processedData = data.toString().toUpperCase();
18            System.out.println("\nWriting to output file...");
19            System.out.println("\nOutput File Content:");
20            System.out.print(processedData);
21        } catch (Exception e) {
22            System.out.println("Error: " + e.getMessage());
23        } finally {
24            scanner.close();
25        }
26    }
27 }
```

Output

```
Enter file content (type END to stop):
Hello World
Welcome to File Handling
END

Input File Content:
Hello World
Welcome to File Handling

Writing to output file...

Output File Content:
HELLO WORLD
WELCOME TO FILE HANDLING

=== Code Execution Successful ===
```

Task 2 – C++ to Python Function Conversion Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

```
main.cpp  Run  Output  Clear

1 #include <iostream>
2 using namespace std;
3
4 void bubbleSort(int arr[], int n) {
5     for(int i = 0; i < n-1; i++) {
6         for(int j = 0; j < n-i-1; j++) {
7             if(arr[j] > arr[j+1]) {
8                 // swap
9                 int temp = arr[j];
10                arr[j] = arr[j+1];
11                arr[j+1] = temp;
12            }
13        }
14    }
15 }
16
17 int main() {
18     int arr[] = {64, 34, 25, 12, 22, 11, 90};
19     int n = sizeof(arr)/sizeof(arr[0]);
20
21     bubbleSort(arr, n);
22
23     cout << "Sorted array: ";
24     for(int i = 0; i < n; i++) {
25         cout << arr[i] << " ";
26     }
27
28     return 0;
29 }
```

Sorted array: 11 12 22 25 34 64 90
=== Code Execution Successful ===

```
main.py  Run  Output  Clear

1 # Python Equivalent of Bubble Sort
2
3 def bubble_sort(arr):
4     n = len(arr)
5
6     for i in range(n - 1):
7         for j in range(n - i - 1):
8             if arr[j] > arr[j + 1]:
9                 # swap
10                arr[j], arr[j + 1] = arr[j + 1], arr[j]
11
12 # Test the function
13 arr = [64, 34, 25, 12, 22, 11, 90]
14
15 bubble_sort(arr)
16
17 print("Sorted array:", end=" ")
18 for num in arr:
19     print(num, end=" ")
```

Sorted array: 11 12 22 25 34 64 90
=== Code Execution Successful ===

Task 3 – SQL to Pandas Query Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
- Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.
- Test the generated code using a sample dataset.

- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

The image shows two side-by-side screenshots of online development environments. The top screenshot is from Programiz's Online SQL Editor. It features a sidebar with database schemas for 'Customers' and 'Orders'. The 'Customers' schema includes fields like customer_id, first_name, last_name, age, and country. The 'Orders' schema includes order_id, item, amount, and customer_id. The main area is labeled 'Input' and contains an SQL query: `SELECT item, SUM(amount) AS total_amount FROM Orders GROUP BY item;`. A 'Run SQL' button is visible. The bottom screenshot is from a Python IDE (likely Replit). It shows a file named 'main.py' with Python code that uses Pandas to create a DataFrame from a dictionary of department and sales data, then groups it by department and sums the sales. The code is:

```
1 import pandas as pd
2
3 data = {
4     "department": ["HR", "IT", "HR", "IT", "Sales", "Sales", "HR"],
5     "sales": [1000, 2000, 1500, 2500, 3000, 3500, 1200]
6 }
7
8 df = pd.DataFrame(data)
9
10 result = df.groupby("department", as_index=False)["sales"].sum()
11 result.rename(columns={"sales": "total_sales"}, inplace=True)
12
13 print(result)
```

 The 'Output' pane on the right shows the result of the execution:

department	total_sales
HR	3700
IT	4500
Sales	6500

. Below the table, it says '=== Code Execution Successful ==='.

Task 4 – Real-Time Application: Multi-Language Snippet Converter Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).

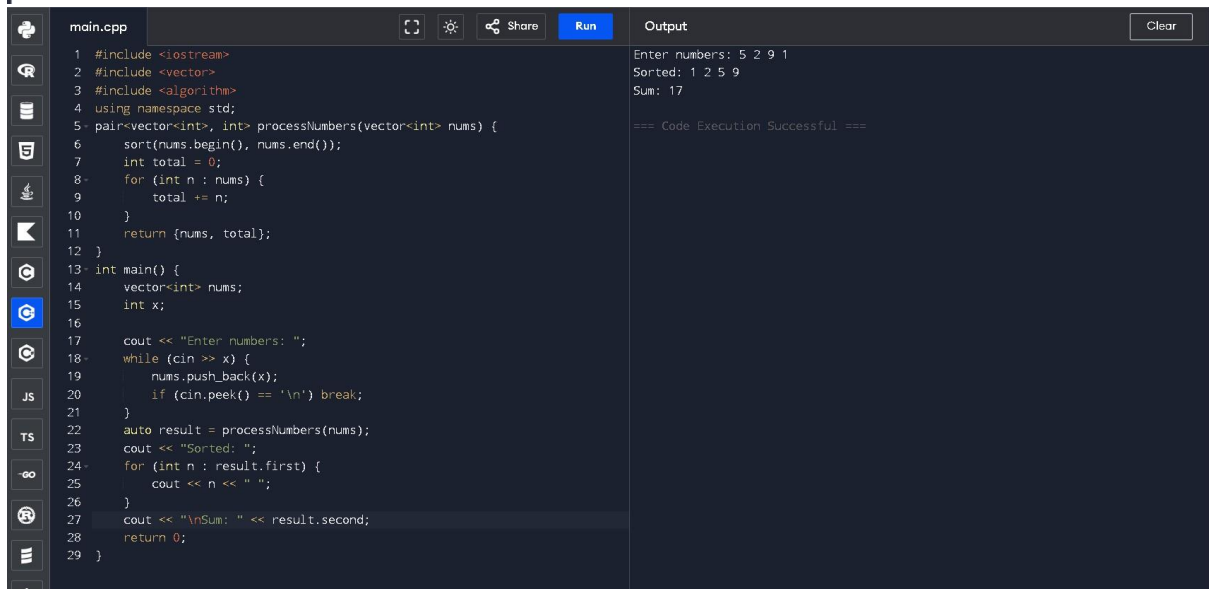
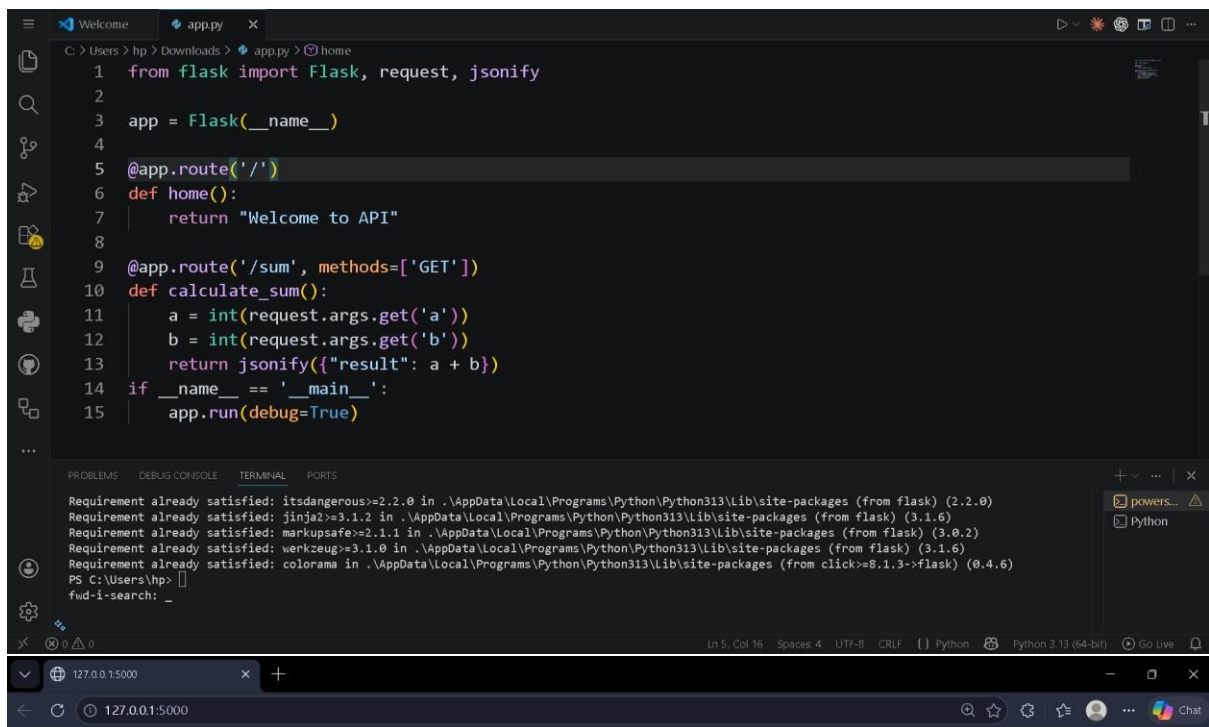
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.



Task -5: (Basic Function Translation (Python to C++))

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

The image displays two screenshots of code editors, likely from an IDE or online platform, showing the translation of a Python function to C++ and their execution results.

Top Screenshot (Python):

- File:** main.py
- Code:**

```
1 def check_even_odd(n):
2     if n % 2 == 0:
3         return "Even"
4     else:
5         return "Odd"
6
7 # Input
8 num = int(input("Enter a number: "))
9
10 # Output
11 result = check_even_odd(num)
12 print("The number is:", result)
```
- Output:**

```
Enter a number: 4
The number is: Even

=== Code Execution Successful ===
```

Bottom Screenshot (C++):

- File:** main.cpp
- Code:**

```
1 #include <iostream>
2 using namespace std;
3
4 // Function to check even or odd
5 string checkEvenOdd(int n) {
6     if (n % 2 == 0)
7         return "Even";
8     else
9         return "Odd";
10 }
11
12 int main() {
13     int num;
14
15     cout << "Enter a number: ";
16     cin >> num;
17
18     string result = checkEvenOdd(num);
19
20     cout << "The number is: " << result << endl;
21
22     return 0;
23 }
```
- Output:**

```
Enter a number: 11
The number is: Odd

=== Code Execution Successful ===
```